



UNIVERSIDAD DE GRANADA

**Departamento de Ciencias de la
Computación e Inteligencia Artificial**

Reto 1: Abstracción

Dpto. Ciencias de la Computación e Inteligencia Artificial
E.T.S. de Ingenierías Informática y de Telecomunicación
Universidad de Granada

Estructuras de Datos

Grado en Ingeniería Informática
Doble Grado en Ingeniería Informática y Matemáticas
Doble Grado en Ingeniería Informática y ADE

El **Juego de la Vida** (Conway's Life) es el ejemplo más famoso de autómatas celulares y fue diseñado por el matemático J. H. Conway en 1970. Una breve descripción sería:

- El “universo” consiste en una matriz bidimensional rectangular de celdas.
- Cada celda puede encontrarse, en un momento dado, en el estado *viva* (1) o *muerta* (0).
- En cada nueva iteración, cada celda puede cambiar de estado dependiendo de cuantas de sus 8 celdas vecinas están vivas:
 - Si una celda *muerta* está rodeada exactamente por 3 celdas *vivas*, pasa al estado *viva*.
 - Si una celda *viva* **no** está rodeada por 2 o 3 celdas *vivas*, pasa al estado *muerta*.
 - En cualquier otro caso la celda no cambia de estado

Como se puede apreciar en realidad el *juego de la vida* no es un “juego” en absoluto, ya que dado un patrón inicial, su evolución está completamente prefijada y no requiere de más datos provenientes del usuario.

El objetivo del reto es hacer una abstracción sobre el problema que conduzca eventualmente a un programa al que se le indique un nombre de fichero (que almacena el patrón de celdas inicial) y un valor numérico n , y que simule la evolución de ese patrón inicial durante n iteraciones aplicando las reglas anteriores, para proporcionar como resultado otro fichero con el patrón final (además de información como su tamaño, el número de celdas vivas y el tiempo empleado).

El tamaño de la matriz donde evolucionan los patrones **no está limitado** (es decir, que se puede ampliar dinámicamente para dar cabida a todas las celdas vivas que se generen durante la evolución del patrón inicial).

La abstracción debe tener en cuenta que el programa que se construya a partir de ella debe ser lo más eficiente posible en cuanto a tiempo y espacio, pero con la restricción de que debe basarse **exclusivamente** en el uso de vectores ó matrices rectangulares y no emplear estructuras de datos o algoritmos avanzados (hashing, árboles, etc.).

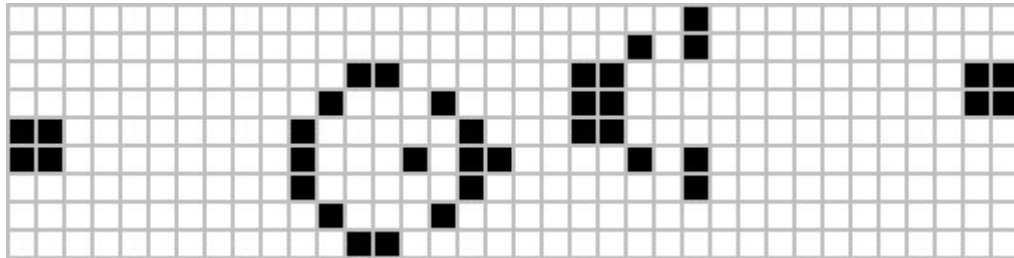
Descripción detallada

Como se ha indicado, el programa resultante de la abstracción recibe como entrada un nombre de fichero y un entero que indica el número de iteraciones. Estas entradas pueden pedirse al usuario (ya sea en modo consola o gráfico) o proporcionarse como parámetros del programa. El nombre del fichero debe hacer referencia a un fichero de texto existente que almacena un rectángulo de celdas (denominado *patrón*) y que tiene el siguiente formato:

- La primera línea contiene un entero que indica el número de filas del patrón, n .
- La segunda línea contiene un entero que indica el número de columnas del patrón, m .
- Le siguen n líneas de texto de m caracteres cada una, los caracteres pueden ser el punto (celda *muerta*) o una X mayúscula (celda *viva*)

Un patrón siempre se indica mediante el rectángulo **más pequeño** que contiene a todas las celdas vivas. Nunca se puede dar el caso de que la primera o última fila, o la primera o última columna estén totalmente formadas por celdas muertas. Atención porque la salida del programa (el patrón final) **debe cumplir esta condición**.

Por ejemplo, el patrón *cañón de deslizadores de Gosper*:



Se representaría mediante el siguiente fichero de texto:

```

9
36
.....X.....
.....X.X.....
.....XX.....XX.....XX
.....X..X...XX.....XX
XX.....X...X...XX.....
XX.....X..X.XX...X.X.....
.....X...X.....X.....
.....X..X.....
.....XX.....

```

Tras recibir estos datos, el programa pasaría a simular el número de iteraciones indicadas aplicando las reglas del juego de la vida. Hay que tener en cuenta que el espacio necesario puede tener que incrementarse a medida que transcurre la simulación: Por ejemplo, el patrón anterior genera *deslizadores* (patrones que se desplazan) y aunque en las etapas iniciales ocupa únicamente una matriz de 8x36 celdas, a partir de un determinado momento cada 4 iteraciones necesita una fila y columna adicional para representar todas las celdas vivas.

Se pueden aplicar optimizaciones sencillas para ahorrar espacio (por ejemplo, reutilizar o tener en cuenta las filas o columnas de los bordes de la matriz que tras una iteración pasen a constar únicamente de celdas muertas) pero no se pueden usar representaciones comprimidas de la matriz u otros métodos avanzados.

Al finalizar la simulación, el programa debería mostrar los siguientes datos: (el orden debe ser exactamente el indicado y cada ítem ocupar una línea distinta)

- Número de iteraciones realizadas
- Número de celdas vivas del patrón final
- Dimensiones (número de filas y columnas) del rectángulo mínimo que contiene a todas las celdas vivas del patrón final.

- Tiempo empleado por el programa.

Por último, pedirá al usuario el nombre del fichero donde se almacenará el patrón final (con el formato descrito anteriormente). Si el usuario introduce una cadena vacía no se almacenará el resultado.

Ejemplo

Si el fichero *exploder.txt* contiene el siguiente patrón:

```
5
5
X.X.X
X...X
X...X
X...X
X...X
X.X.X
```

Un ejemplo de la ejecución del programa podría ser la siguiente (en cursiva datos del usuario):

```
> ./reto1 exploder.txt 100
100 iteraciones
72 celdas vivas
Dimensiones: 13 x 13
0.03 segundos
Fichero de salida: resultado.txt
```

Y el contenido del fichero *resultado.txt* sería:

```
13
13
..XX      XX..
...XX...XX...
X..X.X.X.X..X
XXX.XX.XX.XXX
.X.X.X.X.X.X.
..XXX...XXX..
.....
..XXX...XXX..
.X.X.X.X.X.X.
XXX.XX.XX.XXX
X..X.X.X.X..X
...XX...XX...
..XX      XX..
```

Se pide:

Especificar el **TDA Juego de la vida** indicando el tipo de dato y con qué propiedades funcionales lo dotaríais. La idea no es que construyáis un programa que resuelva el juego. La idea es que especifiquéis el TDA con vistas a construir la clase correspondiente y resolver el juego. Todas las funciones que veáis interesantes para formar parte del TDA las habréis de especificar correctamente (preferiblemente usando Doxygen, aunque no es obligatorio) en el formato que queráis (precondiciones/postcondiciones, o con cláusulas de argumentos, valores de retorno, etc..).

Consideraciones:

1. Podrán hacerse equipos de un máximo de 3 personas para debatir la solución y esa solución que se envíe se valorará con una puntuación igual para cada uno de los 3 miembros del equipo.
2. Los 3 miembros del equipo deberán subir a prado el mismo fichero pdf con la solución. Se sugiere como nombre del fichero reto1.pdf. Al principio del fichero deberá constar el nombre de los miembros del equipo.
3. Todas las soluciones viables se puntuarán con 0.1 para la evaluación continua.
4. Me interesa analizar vuestra capacidad de abstracción, de forma habréis de prestar atención al diseño de las clases, no a su implementación. De hecho no quiero que implementéis ni una sola línea de código. Todo habrá de hacerse a nivel de especificación. No pretendo que le dediquéis mucho tiempo, solo el necesario para pensar a alto nivel en la solución.